МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №2

з курсу: „Системи баз даних”

виконав студент 3-го курсу
групи ДА-11
Сидько Н.С.

КИЇВ
2023

Виконання роботи

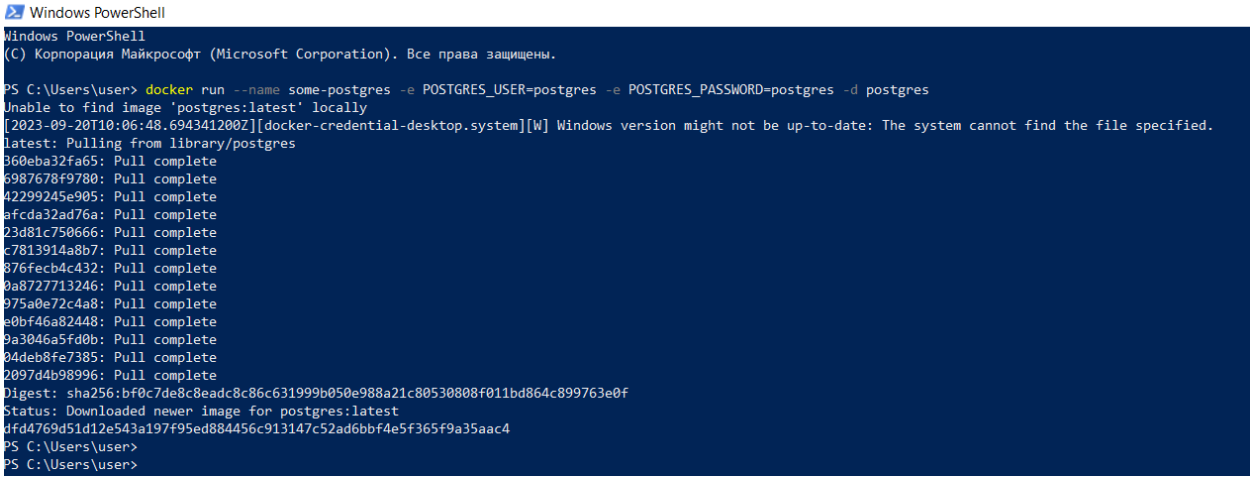
Згідно обраної теми курсової роботи маю **Базу даних обліку успішності в ЗВО.**

PostgreSQL — приклад системи, що слідує традиціям реляційних СБД, її підтримує багато фахівців, вона стабільна й за належного налаштування розв’язує різноманітні завдання.

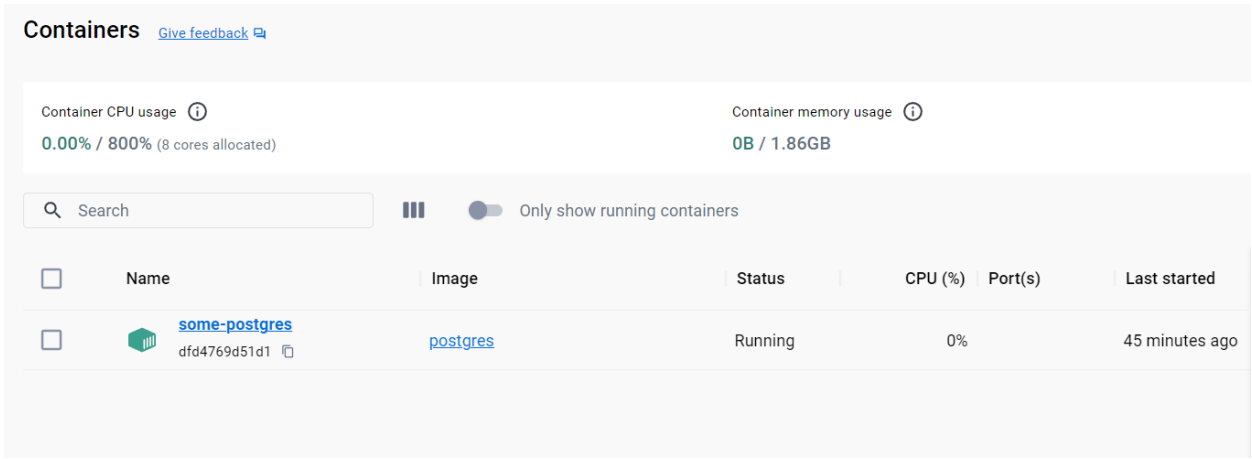
Встановлення СБД PostgreSQL

Для встановлення PostgreSQL використовуємо наступну команду :

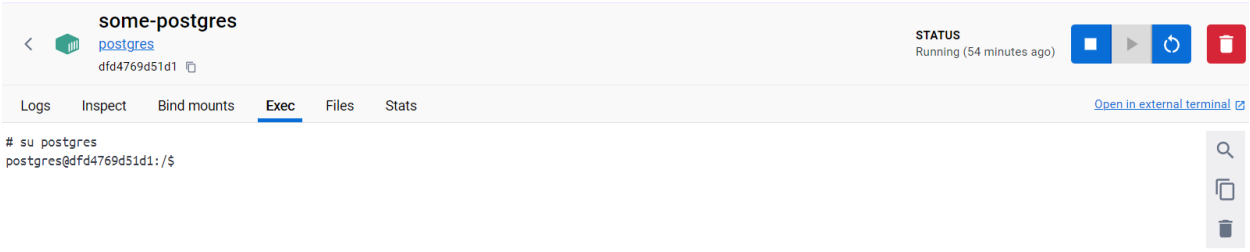
docker run --name some-postgres -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres -d postgres



Після встановлення контейнеру, можна побачити some-postgres у Docker



Відкриваємо консоль контейнера і перемітимось в обліковий запис postgres.



Створення бази даних zvo_info.

Спробуємо створити БД zvo_inf ($ createdb zvo_inf) і увімкнемо консоль ($ psql zvo_inf)

```
postgres@dfd4769d51d1:/$ createdb zvo_inf
postgres@dfd4769d51d1:/$ psql zvo_inf
psql (16.0 (Debian 16.0-1.pgdg120+1))
Type "help" for help.
```

Командами \h і \q – можна подивитися документацію і закрити консоль відповідно.

```
zvo_inf=# \h
Available help:
  ABORT                          ALTER TEXT SEARCH TEMPLATE      CREATE PUBLICATION              DROP FUNCTION                 IMPORT FOREIGN SCHEMA
  ALTER AGGREGATE                ALTER TRIGGER                   CREATE ROLE                     DROP GROUP                    INSERT
  ALTER COLLATION                ALTER TYPE                      CREATE RULE                     DROP INDEX                    LISTEN
  ALTER CONVERSION               ALTER USER                      CREATE SCHEMA                   DROP LANGUAGE                 LOAD
  ALTER DATABASE                 ALTER USER MAPPING              CREATE SEQUENCE                 DROP MATERIALIZED VIEW        LOCK
  ALTER DEFAULT PRIVILEGES       ALTER VIEW                      CREATE SERVER                   DROP OPERATOR                 MERGE
  ALTER DOMAIN                   ANALYZE                         CREATE STATISTICS               DROP OPERATOR CLASS           MOVE
  ALTER EVENT TRIGGER            BEGIN                           CREATE SUBSCRIPTION             DROP OPERATOR FAMILY          NOTIFY
  ALTER EXTENSION                CALL                            CREATE TABLE                    DROP OWNED                    PREPARE
  ALTER FOREIGN DATA WRAPPER     CHECKPOINT                      CREATE TABLE AS                 DROP POLICY                   PREPARE TRANSACTION
  ALTER FOREIGN TABLE            CLOSE                           CREATE TABLESPACE               DROP PROCEDURE                REASSIGN OWNED
  ALTER FUNCTION                 CLUSTER                         CREATE TEXT SEARCH CONFIGURATION DROP PUBLICATION             REFRESH MATERIALIZED VIEW
  ALTER GROUP                    COMMENT                         CREATE TEXT SEARCH DICTIONARY   DROP ROLE                     REINDEX
```

Щоб подивитись довідку щодо SQL треба ввести команду (zvo_inf=# \h CREATE)

```
zvo_inf=# \h CREATE
Command:     CREATE ACCESS METHOD
Description: define a new access method
Syntax:
CREATE ACCESS METHOD name
    TYPE access_method_type
    HANDLER handler_function

URL: https://www.postgresql.org/docs/16/sql-create-access-method.html

Command:     CREATE AGGREGATE
Description: define a new aggregate function
Syntax:
CREATE [ OR REPLACE ] AGGREGATE name ( [ argmode ] [ argname ] arg_data_type [ , ... ] ) (
    SFUNC = sfunc,
```

Створення таблиці

У PostgreSQL таблиці називаються сутностями (TABLE), стовпці — полями (COLUMN), рядки — записами (ROW). Перейдемо до створення таблиці.

```
zvo_inf=# CREATE TABLE faculties ( facult_code char(5) PRIMARY KEY, facult_name text UNIQUE );
CREATE TABLE
```

CRUD — абревіатура основних запитів до баз даних:

- create (дозаписування)
- read (зчитування)
- update (оновлення)
- delete (видалення)

Багато запитів до СБД, крім операторів CREATE, UPDATE, DELETE, виконують зчитування БД оператором SELECT.

Додаємо дані до таблиці

```
zvo_inf=# INSERT INTO faculties (facult_code,facult_name)
zvo_inf-# VALUES ('IASA','Institute Applied System Analysis'),
zvo_inf-# ('MI','Math institute'),
zvo_inf-# ('PI','Physical institute'),
zvo_inf-# ('HI','Institute of History');
INSERT 0 4
```

Перевірка на унікальність

```
zvo_inf=# INSERT INTO faculties (facult_code,facult_name)
zvo_inf-# VALUES ('MI','Math instutite');
ERROR:  duplicate key value violates unique constraint "faculties_pkey"
DETAIL:  Key (facult_code)=(MI   ) already exists.
```

Як бачимо ця умова виконується, тепер зчитаємо таблицю з даними за допомогою SQL запиту SELECT * FROM

```
zvo_inf=# SELECT*FROM faculties;
 facult_code |            facult_name
-------------+-----------------------------------
 IASA        | Institute Applied System Analysis
 MI          | Math institute
 PI          | Physical institute
 HI          | Institute of History
(4 rows)
```

Виконаємо операцію видалення даних з таблиці

```
zvo_inf=# DELETE FROM faculties WHERE fault_code = 'HI';
DELETE 1
zvo_inf=# SELECT*FROM faculties;
 fault_code |            fault_name
------------+-----------------------------------
 IASA       | Institute Applied System Analysis
 MI         | Math institute
 PI         | Physical institute
(3 rows)
```

Створення таблиці students

```
zvo_inf=# CREATE TABLE students (
zvo_inf(# name text NOT NULL,
zvo_inf(# group_code varchar(5) CHECK (group_code <> ''),
zvo_inf(# fault_code char(4) REFERENCES faculties,
zvo_inf(# PRIMARY KEY (group_code, fault_code));
CREATE TABLE
```

Додаємо запис до талбиці **students**

```
zvo_inf=# INSERT INTO students VALUES ('Bob','DA', 'IASA');
INSERT 0 1
zvo_inf=# SELECT*FROM students;
 name | group_code | fault_code
------+------------+------------
 Bob  | DA         | IASA
(1 row)
```

Змінемо значення **group_code**

```
zvo_inf=# UPDATE students SET group_code = 'KA' WHERE name = 'Bob';
UPDATE 1
zvo_inf=# SELECT*FROM students;
 name | group_code | fault_code
------+------------+------------
 Bob  | KA         | IASA
(1 row)
```

Виконаємо **INNER JOIN** (внутрішнє з’єднування students та faculties)

```
zvo_inf=# SELECT students.*,fault_name
zvo_inf-# FROM students INNER JOIN faculties
zvo_inf-# ON students.fault_code = faculties.fault_code;
 name | group_code | fault_code |            fault_name
------+------------+------------+-----------------------------------
 Bob  | KA         | IASA       | Institute Applied System Analysis
(1 row)
```

Внутрішнє з’єднування об’єднує таблиці students і faculties за рядками, у яких значення стовпців fault_code ідентичні. У відповідь на запит СБД сформує

тимчасову проєкцію зі значень усіх стовпців таблиці students. * і стовпця facult_code таблиці faculties.

Створення таблиці Buildings

```
zvo_inf=# CREATE TABLE Buildings(
zvo_inf(# Build_ID SERIAL PRIMARY KEY,
zvo_inf(# Name varchar(100),
zvo_inf(# Adress text,
zvo_inf(# facult_code char(4),
zvo_inf(# group_code varchar(5),
zvo_inf(# FOREIGN KEY (facult_code,group_code) REFERENCES students(facult_code, group_code) MATCH FULL);
CREATE TABLE
```

Додамо значення в таблицю. Для зчитування в консоль значення поля екземпляра, дозаписаного в БД, запит потрібно завершити оператором RETURNING.

```
zvo_inf=# INSERT INTO Buildings (Name, facult_code, group_code) VALUES ('Building number 35','IASA','KA') RETURNING Build_ID;
 build_id
----------
        1
(1 row)

INSERT 0 1
```

Бачимо, що СБД повернув відповідь зі значенням ідентифікатора build_id створеного екземпляра даних.

З’єднаємо таблиці **students** і **buildings**, зазначивши стовпці, які формують складений ключ.

Для скорочення написання SQL-запитів, назвам таблиць встановлюють у відповідність змінні після дійсних найменувань і необов’язкового оператора **AS** (наприклад, **venues v** чи **venues AS v**).

```
zvo_inf=# SELECT b.Build_ID, b.Name, s.name
zvo_inf-# FROM Buildings AS b INNER JOIN students AS s
zvo_inf-# ON b.facult_code = s.facult_code AND b.group_code = s.group_code;
 build_id |        name        | name
----------+--------------------+------
        1 | Building number 35 | Bob
(1 row)
```

Створення таблиці lessons

```
zvo_inf=# CREATE TABLE lesson(
zvo_inf(# title varchar(15),
zvo_inf(# Build_ID integer REFERENCES Buildings,
zvo_inf(# lesson_id SERIAL);
CREATE TABLE
```

Заповнюємо таблицю **lessons** значеннями

```
zvo_inf=# INSERT INTO lesson (title, build_id) VALUES
zvo_inf-# ('History',1),
zvo_inf-# ('Math',NULL),
zvo_inf-# ('Economy',NULL);
INSERT 0 3
```

Виконаємо **LEFT JOIN** з таблицями **lesson** та **Buildings**

```
zvo_inf=# SELECT l.title, b.name FROM lesson AS l LEFT JOIN Buildings AS b ON l.lesson_id = b.build_id;
  title  | name
---------+------
 History |
 Math    |
 Economy |
(3 rows)
```

Виконаємо **RIGHT JOIN** з таблицями **lesson** та **Buildings**

```
zvo_inf=# SELECT l.title, b.name FROM lesson AS l RIGHT JOIN Buildings AS b ON l.lesson_id = b.build_id;
 title |        name
-------+--------------------
       | Building number 35
(1 row)
```

Виконаємо **FULL JOIN** з таблицями **lesson** та **Buildings**

```
zvo_inf=# SELECT l.title, b.name FROM lesson AS l FULL JOIN Buildings AS b ON l.lesson_id = b.build_id;
  title  |        name
---------+--------------------
 History |
 Math    |
 Economy |
         | Building number 35
(4 rows)
```

Створення індексів

Індекс – спеціалізована структура даних, що усуває необхідність повного перебору екземплярів даних під час виконання запиту по БД.

```
zvo_inf=# CREATE INDEX lesson_index ON lesson USING hash(lesson_id);
CREATE INDEX
```

```
zvo_inf=# SELECT*FROM lesson WHERE Build_ID = 1;
  title  | build_id | lesson_id
---------+----------+-----------
 History |        1 |         4
(1 row)
```

Для побудови B-дерева за сумою операцій створимо відповідний індекс

```
zvo_inf=# CREATE INDEX lesson_building ON lesson USING btree (Build_ID);
CREATE INDEX
```

СБД автоматично створює індекс за стовпцями з умовою FOREIGN KEY і для його зчитування введемо **\di**

```
zvo_inf=# zvo_inf#\di
                        List of relations
 Schema |           Name            | Type  |  Owner   |   Table
--------+---------------------------+-------+----------+-----------
 public | buildings_pkey            | index | postgres | buildings
 public | faculties_facult_name_key | index | postgres | faculties
 public | faculties_pkey            | index | postgres | faculties
 public | lesson_building           | index | postgres | lesson
 public | lesson_index              | index | postgres | lesson
 public | students_pkey             | index | postgres | students
(6 rows)
```

Практичні завдання

1. Обчисліть кількість екземплярів даних окремо за всіма таблицями (відповідна «агрегатна» функція є в документації СБД).

```
zvo_inf=# SELECT COUNT(*) FROM faculties;
 count
-------
     3
(1 row)

zvo_inf=# SELECT COUNT(*) FROM students;
 count
-------
     1
(1 row)

zvo_inf=# SELECT COUNT(*) FROM Buildings;
 count
-------
     1
(1 row)

zvo_inf=# SELECT COUNT(*) FROM lesson;
 count
-------
     3
(1 row)
```

2. Зчитайте значення поля group_code (таблиця students) за значенням facult_code

```
zvo_inf=# SELECT students.group_code
zvo_inf-# FROM students
zvo_inf-# WHERE students.facult_code = 'IASA';
 group_code
------------
 KA
(1 row)

zvo_inf=# SELECT*FROM students;
 name | group_code | facult_code
------+------------+-------------
 Bob  | KA         | IASA
(1 row)
```

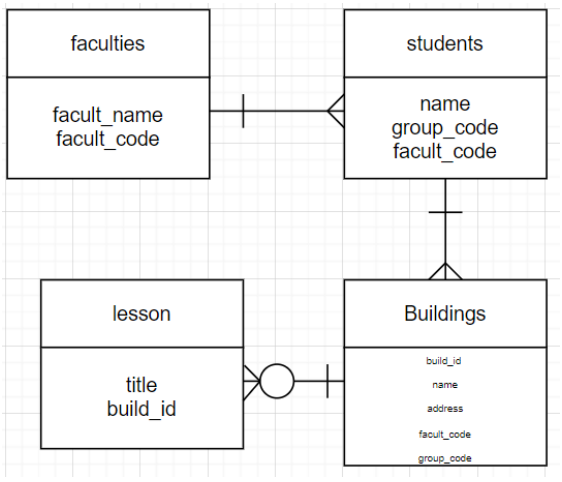
Бачимо що зчитано вірно.

3. Доповніть наявну таблицю students булевим стовпцем students_private із попередньо встановленим значенням TRUE.

```
zvo_inf=# SELECT*FROM students;
 name | group_code | facult_code
------+------------+-------------
 Bob  | KA         | IASA
(1 row)

zvo_inf=# ALTER TABLE students
zvo_inf-# ADD students_private BOOLEAN DEFAULT TRUE;
ALTER TABLE
zvo_inf=# SELECT*FROM students;
 name | group_code | facult_code | students_private
------+------------+-------------+------------------
 Bob  | KA         | IASA        | t
(1 row)
```

Діаграмма зв’язків сутностей



Висновок

У ході виконання лабораторної роботи №2 було виконано роботу з СБД PostgreSQL. Було створено базу даних ЗВО (zvo_inf) та декілька таблиць Були використані CRUD запити, завдяки яким були створені певні об’єднання таблиць (faculties, students, Buildings, lesson) у яких були використані різні типи даних, ключі, оператори. Також було проаналізовано можливості цієї СБД.